

Reinforcement Learning

Planning and Learning with Tabular Methods: Part II

Min-hwan Oh

Seoul National University

Trajectory Sampling

- **Exhaustive sweep**: classical approach from DP: perform sweeps through the entire state space, updating each state once per sweep
- Sample from the state space according to some distribution
 - ▶ Uniform sample: Dyna-Q

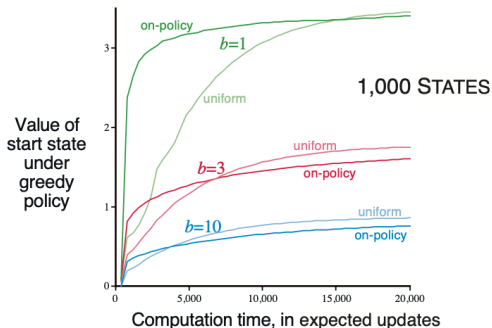
Trajectory Sampling

- **Exhaustive sweep**: classical approach from DP: perform sweeps through the entire state space, updating each state once per sweep
- Sample from the state space according to some distribution
 - ▶ Uniform sample: Dyna-Q
 - ▶ **Trajectory sampling**: On-policy distribution following the current policy
 - ▶ Sample state transitions and rewards are given by the model, and sample actions are given by the current policy.
 - ▶ Simulate explicit individual trajectories and perform update at the state encountered along the way → trajectory sampling

On-policy Distribution

Is the on-policy distribution a good one?

- **Good**: it causes vast, uninteresting parts of the space to be ignored
- **Bad**: it may update the same parts over and over



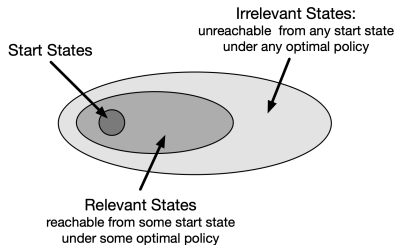
- b : branching factor, results averaged over 200 sample tasks with 1000 states

Real-time Dynamic Programming (RTDP)

- **RTDP**: on-policy trajectory-sampling version of value-iteration
- Updates the values of states visited in actual or simulated trajectories by means of expected tabular value-iteration updates.
- Example of an **asynchronous** DP algorithm
 - ▶ Asynchronous algorithms do not do full sweeps, they update state values in any order.
 - ▶ In RTDP, update order is dictated by the order states are visited in real or simulated trajectories

RTDP Policy Evaluation

- Trajectories start from a designated set of start states
- On-policy trajectory sampling allows to focus on useful states



RTDP Policy Improvement

- There might be states that cannot be reached by any of the optimal policies from any of the start states.
- No need to specify optimal actions for those irrelevant states
- Only need an **optimal partial policy**!

RTDP Policy Improvement

- There might be states that cannot be reached by any of the optimal policies from any of the start states.
- No need to specify optimal actions for those irrelevant states
- Only need an **optimal partial policy**!

But...

- Finding such an optimal partial policy can require visiting all state-action pairs, even those turning out to be irrelevant in the end

RTDP Policy Improvement

- There might be states that cannot be reached by any of the optimal policies from any of the start states.
- No need to specify optimal actions for those irrelevant states
- Only need an **optimal partial policy**!

But...

- Finding such an optimal partial policy can require visiting all state-action pairs, even those turning out to be irrelevant in the end

However,

- For certain problems, RTDP is guaranteed to find an optimal policy without visiting every state infinitely often, or even without visiting some states at all.

RTDP: Conditions for Guarantee of Optimal Convergence

[Proved by Barto, Bradtke, and Singh (1995)]

- Undiscounted episodic task for MDP with absorbing goal state that generate zero reward
- Initial value of every goal state is 0.
- There exists at least one policy that guarantees a goal state will be reached from any start state.
- All rewards from transitions from non-goal states are strictly negative.
- The initial values are greater than (or equal to) optimal values

Planning at Decision Time

- Background planning

- ▶ In Dyna (or DP), planning gradually improves a policy or value function on the basis of simulated experience from a model
- ▶ Well before an action is selected for any current state S_t , planning has played a part in improving the way to select the action for many states.
- ▶ Planning here is not focused on the current state

Planning at Decision Time

- **Background planning**

- ▶ In Dyna (or DP), planning gradually improves a policy or value function on the basis of simulated experience from a model
- ▶ Well before an action is selected for any current state S_t , planning has played a part in improving the way to select the action for many states.
- ▶ Planning here is not focused on the current state

- **Decision-time planning**

- ▶ Begin and complete planning after visiting each state S_t to produce A_t
- ▶ Planning when only state-values are available
- ▶ At each state we select an action based on the values of model-predicted next states for each action
- ▶ Can also look much deeper than one step ahead
- ▶ Planning focuses on a particular state.

Decision-time Planning

Even when planning is done at decision time,

- can still view it as proceeding from simulated experience to updates and values, and ultimately to a policy.
- It's just that now the values and policy are specific to the current state.

In general, we may want to do a bit of both:

- Focus planning on the current state
- Store the results of planning so as to help when we return to the same state after

Decision-time planning is best when fast responses are *not* required.

Heuristic Search

- For each state encountered, a large tree of possible continuations is considered.
- The approximate value function is applied to the leaf nodes and then backed up toward the current state at the root.
- The best is chosen as the current action.
- All other backed-up values are discarded.

Heuristic Search

- For each state encountered, a large tree of possible continuations is considered.
- The approximate value function is applied to the leaf nodes and then backed up toward the current state at the root.
- The best is chosen as the current action.
- All other backed-up values are discarded.

An extension of the idea of a greedy policy beyond a single step to obtain better action selections

- Effectiveness of heuristic search is due to its search tree being tightly focused on the states and actions that might immediately follow the current state.
- Tradeoff here is that deeper search leads to more computation

Heuristic Search Illustration

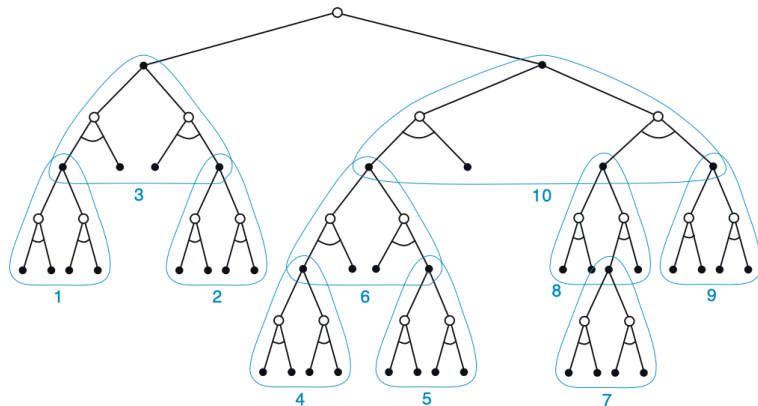


Figure: Heuristic search as a sequence of one-step updates (in blue), backing up values from leaf to root. The ordering here is selective depth-first search.

Rollout Algorithms

- Decision-time planning algorithms based on Monte Carlo control applied to simulated trajectories.
- Trajectories all begin at the current environment state.
- Estimate action values for a given policy by averaging the returns of many simulated trajectories
- Unlike MC algorithm, the goal here is not to estimate a complete optimal action-value function.
- Only care about the **current state** and a **given policy**, called the **rollout policy**.
 - ▶ No need to sample outcomes for every state-action pair, and no need to approximate a function over either the state (or state-action) space
- The aim of a rollout algorithm is to improve upon the rollout policy, not to find an optimal policy

Rollout Algorithms in Practice

- May run many trials in parallel on separate processes because Monte Carlo trials are independent of one another
- Can truncate the trajectories short of complete episodes and correct the truncated returns by the means of a stored evaluation function (does this remind you of something?)
- Monitor the Monte Carlo simulations and prune away actions that are unlikely to be the best

Monte Carlo Tree Search (MCTS)

- Decision-time planning
- Essentially a rollout algorithm, but enhanced with a means of accumulating value estimates from the Monte Carlo simulations
- Successively direct simulations towards more highly-rewarding trajectories
- Used in AlphaGo!

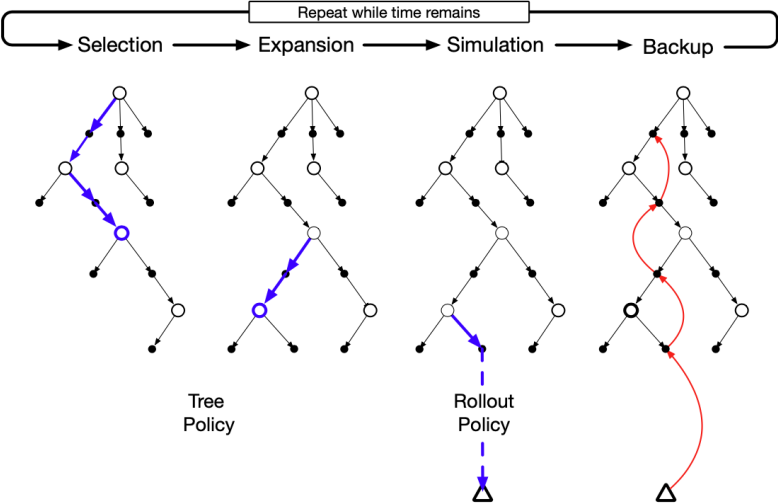
Monte Carlo Tree Search (MCTS) cont'd

- MCTS is executed after encountering each new state to select the action
- Each execution is an iterative process that simulates many trajectories starting from the current state
- The core idea of MCTS is to successively focus multiple simulations starting at the current state by extending the initial portions of trajectories that have received high evaluations from earlier simulations
- It benefits from online, incremental, sample-based value estimation and policy improvement.
- Focuses the Monte Carlo trials on trajectories whose initial segments are common to high-return trajectories previously simulated

How MCTS works

- MCTS incrementally extends the tree by adding nodes representing states that look promising based on the results of the simulated trajectories.
- Any simulated trajectory will pass through the tree and then exit it at some leaf node.
- Outside the tree and at the leaf nodes, a simple **rollout policy** is used for action selections.
- Inside the tree, we have value estimates for at least some of the actions, so we can pick among them using an informed policy, called the **tree policy**, that balances exploration and exploitation, e.g., ϵ -greedy or UCB selection

MCTS Illustration



Next class: On-policy Prediction with Approximation
(Chapter 9 of Sutton & Barto)